ResearchPilot 研途智控 —— 项目文档

科研任务智能管理 Web 应用：把大目标拆成小任务，延期了自动帮你重新规划。

项目	内容
项目名称	ResearchPilot（研途智控）
文档版本	v1.1（2026-10-12）
技术栈	React 19 + TypeScript + Vite 7（前端）· FastAPI 0.143 + Python 3.12（后端）· SQLite（存储）· DeepSeek deepseek-chat（AI 能力）
代码仓库	本地 Git 仓库，5 次提交，含完整演进历史
核心创新	本地风险预警引擎 + DeepSeek 动态重规划（原计划 vs 新计划对比、历史留档）
AI 协作方式	Claude Code 负责工程实现，DeepSeek 提供业务大模型能力，风险计算等基础逻辑为本地规则引擎

目录

1. 项目概述、痛点与应用价值
2. 需求分析与用户场景
3. 意图控制策略（重点）：如何用 Prompt 约束 Claude 写代码、约束 DeepSeek 输出 JSON
4. 系统架构（前端、FastAPI 后端、SQLite、DeepSeek API）
5. 核心功能：任务生成、风险计算、重规划逻辑
6. AI 协同开发演进：3 组完整 Prompt 案例
7. 测试结果
8. 可复现性说明（Git、README、环境配置）
9. 不足与未来优化方向

---

1. 项目概述、痛点与应用价值

1.1 项目概述

ResearchPilot（研途智控）是面向硕/博研究生的科研任务智能管理 Web 应用。它解决一个朴素而普遍的问题：**导师只给一个大目标，学生不知道如何把它变成"每天能干的事"；而一旦延期，又往往因为不知道怎么调整而直接放弃。**系统以"项目 → 任务"两级结构组织科研工作，围绕一条完整闭环设计：

```
新建项目（目标/截止时间/每周可用小时）
   → AI 拆解科研计划（DeepSeek 生成任务链，预览确认后写入看板）
   → 日常更新进度（看板流转 + 进度记录 + 投入小时）
```

```
→ 风险实时预警（本地规则引擎：逾期/阻塞/落后/工时不足）
→ AI 动态重规划（一键生成调整方案，新旧对比，采纳后自动更新并留档）
→ 汇报输出（组会周报 / 导师汇报自动生成）
```

此外还提供两个"科研刚需"附加能力：**文献精读与引用溯源助手**（上传 PDF、提问时只依据原文回答并给出可点击的原文引用）与**汇报中心**（从数据库自动聚合数据生成组会周报/导师汇报）。

1.2 核心痛点与解法

痛点	现状	本项目的解法
拆解难	导师只给大目标，不知如何拆成每天能干的事	AI 科研计划生成：输入研究主题/截止时间/每周可用小时，DeepSeek 自动拆解为带依赖关系的任务链，预览确认后写入看板
发现晚	任务延期后知后觉，临近截止才发现来不及	本地风险预警引擎：实时计算逾期/前置阻塞/进度落后/剩余工时不足，输出可解释原因，不消耗 AI token
调整难	延期后不知如何重排优先级，往往直接放弃	DeepSeek 动态重规划（核心创新）：一键生成调整方案，原计划 vs 新计划对比高亮，接受后自动更新任务并留档
无监督	没有进度反馈，动力不足	三列看板、进度记录、仪表盘图表（趋势/状态分布/风险提醒）
汇报累	组会/导师汇报临时凑内容	汇报中心：自动聚合本周数据，生成组会周报/导师汇报（Markdown，可编辑导出）

1.3 应用价值

对用户（研究生本人）：把模糊目标变成可执行计划（任务链自带日期排布、耗时估算、优先级与前置依赖）；把风险消灭在"还来得及"的时候（可解释的原因 + 一键重规划，让"延期"从终点变成一次可管理的调整）；时间可量化（投入小时、进度曲线全部落库）；汇报零成本（组会前 10 秒生成）。

对工程实践（AI 协同开发方法论）：本项目全程由 AI 协同开发完成，沉淀了一套可复用的**意图控制策略**（第 3 章）；"演示模式 + 本地规则引擎"的设计使系统**不依赖任何 API Key 也能完整运行**，可作课程演示/快速体验/断网环境的兜底。

1.4 功能总览

模块	功能	依赖 AI
项目/任务管理	项目 CRUD、任务两级结构（里程碑+任务）、三列看板、状态流转、进度记录	否
仪表盘	统计卡片、完成趋势图、状态分布、风险提醒	否
AI 科研计划	目标拆解为 6~12 个带依赖的任务，预览 → 确认写入	是（无 Key 时演示模式）
风险预警	逾期/前置阻塞/进度落后/工时不足，任务级+项目级，可解释	否（纯本地规则）
动态重规划	生成调整方案 → 新旧对比 → 采纳/放弃 → 历史留档	是（无 Key 时规则模拟）

模块	功能	依赖 AI
文献库	PDF 上传、500~800 字切块、关键词检索、引用溯源问答、笔记	是（无 Key 时检索直出）
汇报中心	组会周报/导师汇报生成、编辑保存、复制、导出 .md	是（无 Key 时模板生成）
周报	近 7 天活动聚合 + AI 周报正文	是（无 Key 时模板生成）

2. 需求分析与用户场景

2.1 目标用户画像

维度	描述
身份	硕/博研究生（以理工科为主，兼顾文科研一~研三）
科研周期	3 个月（小论文）~ 3 年（学位论文）
每周可用时间	5~20 小时（被课程/助教/横向项目挤压）
典型困境	目标模糊不会拆解、延期后知后觉、缺乏进度反馈、汇报前临时凑内容
使用环境	个人电脑单机使用，无多用户协作需求

2.2 核心需求清单

编号	需求	优先级
R1	输入研究主题、截止日期、每周可用小时，自动生成带依赖关系与日期排布的任务计划	高
R2	计划生成后先预览，用户可确认/重新生成，确认后才写入数据库	高
R3	任务延期/被阻塞/工时不足时实时给出可解释的风险提示	高
R4	延期后一键生成重规划方案，原计划与新计划对比展示，采纳后自动更新任务，全部留档可回溯	高
R5	项目、任务、进度记录本地持久化，刷新/重启不丢数据	高
R6	文献 PDF 上传后可就内容提问，答案必须可溯源到原文	中
R7	一键生成组会周报/导师汇报，可编辑、可导出	中
R8	未配置 API Key 时全功能可演示运行	中
R9	AI 调用失败（超时/余额不足/Key 无效等）给出中文友好提示，页面不崩溃	中

2.3 用户场景

场景一：开题后不知道怎么开始（R1、R2）——小李拿到题目"虚拟世界智能体行为规划，12 周完成论文初稿"，每周能投入 10 小时，但不知道从哪下手。他新建项目并填写研究主题、截止日期（今天+12 周）、每周 10 小时、当前阶段"文献调研"，点击【生成科研计划】。约 10~60 秒后，系统展示 9 个任务的预览：从"完成文献综述初稿"到"撰写论文初稿"，每个任务带起止日期、预计耗时、优先级、前置依赖和安排理由，总耗时 102 小时（正好等于 10 小时 × 12 周 × 0.85 缓冲系数）。确认后任务落入三列看板，今天该做什么一目了然。

场景二：实验拖期，眼看要来不及（R3、R4）——第 8 周，"核心实验"任务已逾期 2 天、进度只有 40%，后面的"论文撰写"被它阻塞。风险预警区标红并给出原因："已逾期 2 天，进度仅 40%"、"前置任务「核心实验」尚未完成，当前任务被阻塞"。小李点击【重新规划】，DeepSeek 生成方案：核心实验顺延至 10-31、论文撰写顺延至 11-01~11-08、答辩准备压缩至 11-08~11-11，并指出"总工时需求约 58 小时，超出剩余可用工时约 18 小时，建议提高每周投入"。小李在新旧对比表中看到每处改动的删除线旧值和高亮新值，点击【接受调整】，看板自动更新，本次调整连同原因一起存入历史记录。

场景三：组会前一小时（R7）——小李打开"汇报中心"，选择近 7 天和"组会周报"类型，点击生成。系统从数据库聚合本周完成的任务、进度记录、风险与重规划历史，交给 DeepSeek 生成五章节周报（本周完成/未完成及原因/当前风险/下周计划/需要导师支持）。其中"需要导师支持"一节甚至从进度备注"数据格式问题卡住"中推断出"需导师协调数据来源"。小李编辑两处细节后复制进组会文档，全程 5 分钟。

场景四：精读论文时找不到依据（R6）——小李上传一篇 PDF 到文献库，提问"研究方法是什么"。系统用 jieba 分词检索出最相关的 5 个片段交给 DeepSeek，回答列出三种研究方法（问卷调查/实验对照/用户访谈），每条结论带引用按钮，点击自动滚动定位到原文片段并高亮。回答置信度 95%，且明确保证"只依据原文回答，片段中找不到就说找不到"。（无 Key 时的演示模式场景见 8.5 节。）

2.4 主线使用流程

```
首次使用：新建项目（目标/截止时间/每周小时）→ AI 拆解预览 → 确认写入看板
日常使用：看板流转任务 → 更新进度（进度+投入小时+备注）→ 仪表盘看趋势
风险处理：风险预警区查看原因 → 一键重规划 → 对比新旧计划 → 采纳/放弃（留档）
汇报输出：汇报中心选时间范围与类型 → 生成 → 编辑 → 复制/导出
```

2.5 MVP 范围决策（需求阶段定稿）

#	决策	理由
1	不做登录注册，单用户本地使用	个人工具定位，后期可加
2	AI 拆解先预览、改完再写入	防止 AI 结果直接污染数据
3	任务两级结构（里程碑 + 任务）	满足"大目标 → 小任务"的拆解语义
4	风险识别用本地规则，不消耗 AI	可解释、零 token 成本、断网可用
5	看板用按钮操作，不做鼠标拖拽	拖拽交互成本高，列入后期
6	不做邮件/推送通知，打开应用看提醒	单机场景下推送价值有限

3. 意图控制策略（重点）

> 本章回答一个问题：**当"写代码的"和"出内容的"都是大模型时，如何保证它们稳定地按你的意图工作?**
> 本项目中 Claude Code 负责工程实现（需求分析、前后端代码、测试与排障），DeepSeek 负责业务内容生成（任务拆解、重规划、文献问答、汇报撰写）。对两者采用不同策略：对 Claude 用**需求规格化 + 边界护栏 + 强制验收**，对 DeepSeek 用**模板化 Prompt + 强约束 + 代码层四道防线**。

3.1 问题背景：大模型的两个不确定性

1. **Claude 的不确定性在"范围"**：自然语言需求天然模糊，"帮我做个任务管理"和"帮我做个任务管理，要求如下 7 条，第 8 条不要改动已有代码"产出的是两个级别的工程。更危险的是主动性：没有约束时 AI 可能重写无关模块、更换技术栈、添加没人要的功能。
2. **DeepSeek 的不确定性在"格式"**：大模型输出是概率采样，即使"要求返回 JSON"，实际可能返回带解释文字的 JSON、Markdown 代码块包裹的 JSON、字段缺失或类型错误的 JSON，甚至直接道歉说"抱歉我无法输出 JSON"。而这些输出要被 `json.loads` 消费并写入数据库——**格式不可靠等于功能不可用**。

两套策略叠加第三层"代码防线"，构成完整体系：

```
用户意图（自然语言需求）
   │  第一层：需求规格化 Prompt（边界护栏 + 验收标准 + 执行顺序约束）
   ▼
Claude Code ── 产出工程代码（前端/后端/测试/文档）
   │  第二层：模板化 Prompt（角色 + 格式契约 + 逐条约束 + 温度调低）
   ▼
DeepSeek API ── 产出结构化 JSON / Markdown
   │  第三层：代码防线（容错解析 → 逐字段校验 → 自动重试 → 演示模式兜底）
   ▼
Pydantic/SQLAlchemy 落库 → 前端展示（用户确认后才写入）
```

3.2 约束 Claude Code 的策略：让"写代码的 AI"不跑偏

从本项目 3 组 Prompt 案例（第 6 章）提炼出 7 个有效手法：

手法 1：给出完整规格，而不是模糊描述。需求 Prompt 从不只写"做一个 AI 拆解功能"，而是同时给出输入、输出结构、行为与验收：输入定义（研究主题/截止日/每周小时/当前阶段/已完成工作）；输出契约（9 个字段的 JSON 结构直接写进需求，"强制模型返回严格 JSON 格式"）；行为定义（7 条编号需求：密钥不出后端、重试 2 次、预览确认才入库、演示模式、异常捕获……）；验收标准（"完成后给我测试案例，测试文本：……"——把测试素材也放进需求，AI 交付时必须跑给你看）。

手法 2：边界护栏——明确"不许做什么"。每轮迭代都带护栏条款，防止 AI 的主动性破坏已有成果：

护栏	需求中的原话
保护已有代码	"不要改动已经写好的任务 CRUD 代码"（案例二、三重复出现）
保护技术栈	"先扫描项目结构，不要重写整个项目，不要更换技术栈"
保护密钥	"密钥严禁暴露到前端""API Key 只存在于后端 .env"
保护职责边界	"风险判断逻辑写在后端代码，不要交给大模型做基础计算；大模型只负责给出调整建议"

事后验证：重规划落地后"原有任务生成与 CRUD 代码零改动"成为交付说明的固定检查项，冒烟测试 36/36 承担"护栏有没有生效"的回归验证。

手法 3：先方案、后代码（大功能）。对文献库、汇报中心这类多文件功能，需求里直接要求"**请先列出你要新增/修改的文件，等我确认后再写代码**"。AI 先交付文件清单 + 接口设计 + 关键设计点（如扫描版检测阈值、检索打分策略、JSON 输出契约），确认后才动工——把"设计评审"前置，返工成本从"改代码"降为"改方案"。两轮方案确认后，实际开发均一次通过测试。

手法 4：强制测试与验收，且测试素材由用户指定。需求末尾固定要求"完成后给我测试流程/测试案例"。AI 的交付因此不是"代码写完"，而是"代码写完 + 三个实例（演示模式/真实 Key/错误 Key）跑通 + 冒烟回归通过 + 给你浏览器操作步骤"。案例二中真实 Key 测试暴露了 `.env` 里 `DEEPSEEK_BASE_URL` 填成完整端点导致 404 的配置错误——没有"用真实 Key 测"这条，这个 bug 会留到用户手里才爆炸。

手法 5：约束执行顺序，避免环境级事故。案例二中 Claude 动手前自查："你的 8000 后端带 --reload，新代码会 import httpx，所以先装依赖再写代码，避免重载崩溃。"更普适的写法是直接声明："新增依赖时先安装并验证，再改代码。"

手法 6：把降级设计写进需求（演示模式前置）。三个 AI 功能的需求里都有同样一条："**没有 API Key 时进入演示模式**（返回预设/规则模拟/模板内容），并明确标记。"价值：让"AI 是增强不是必需"成为架构原则；测试环境与演示环境天然可用；AI 失败永远有兜底路径。

手法 7：验收标准可量化。"36 项冒烟测试全过""前端 build 通过""原计划 vs 新计划对比表"——可检查、可数字化的验收项让"做完没做完"没有争议空间。

小结：对 Claude 的约束本质上是**把人脑子里的"规格 + 边界 + 验收"翻译成 AI 无法曲解的形式**。效果：3 个大功能模块需求一次性实现通过测试；两轮"先方案后代码"的功能零返工；全部迭代未发生一次"动了不该动的代码"事故。

3.3 约束 DeepSeek 的策略：让"出内容的 AI"输出可被程序消费的 JSON

DeepSeek 的输出要被 `json.loads` 解析、被 Pydantic 校验、被写入 SQLite。因此 Prompt 的目标不是"让它答得好"，而是"让它输出的每个字节都在契约内"。四个生成场景的 Prompt 契约：

场景	输出契约	关键约束
科研计划生成	JSON 数组（任务对象 ×6~12）	10 条约束，见 3.3.1
动态重规划	JSON 对象 `{plan_content, impact_summary, changes[]}`	7 条约束，见 3.3.2
文献问答	JSON 对象 `{answer, citations[], confidence}`	5 条规则（含反幻觉），见 3.3.3
汇报/周报	Markdown 正文	章节锁定 + 反编造，见 3.3.4

3.3.1 科研计划生成 Prompt 逐条解析

发送给 DeepSeek 的 system prompt（`backend/app/services/ai_planner.py` 组装，实际生效文本）：

```
你是资深科研项目规划专家。根据用户提供的科研信息，制定一份可执行的科研任务计划，
覆盖从当前阶段到项目截止日期的完整路径。
输出要求（必须严格遵守）：
1. 只输出一个 JSON 数组，不要输出任何解释文字、前后缀、Markdown 代码块标记；
2. 数组每个元素是一个任务对象，字段如下（示例）：
   {"task_id": "T1", "title": "完成文献综述初稿", "description": "系统梳理智能体行为规划相关文献",
    "start_date": "2026-10-13", "due_date": "2026-10-26", "priority": "高",
    "expected_deliverable": "文献综述初稿（3000字）", "dependencies": [], "estimated_hours": 18,
    "rationale": "文献调研是当前阶段的首要工作，为方法设计打基础"}
3. task_id 为唯一字符串（如 T1、T2…），dependencies 只能引用数组中已出现的 task_id，
   没有前置任务则为空数组 [];
4. priority 只能是「高」「中」「低」之一；
5. estimated_hours 为数字（小时）；
6. 任务数量 6~12 个；
7. 所有日期必须在 {today} 到 {deadline} 之间，且 due_date 不得早于 start_date；
8. 全部任务 estimated_hours 总和约 {total} 小时（每周 {X} 小时 × {N} 周 × 0.85 缓冲系数）；
9. 从当前研究阶段之后开始安排任务，已完成的工作不再重复安排；
10. 任务安排要有前后依赖关系，形成一条从当前阶段到论文初稿的推进链路。
```

逐条解读这条 Prompt 的设计意图：

条目	手法	防的是什么
开头角色设定	人设锚定（"资深科研项目规划专家"）	回答风格漂移、拒答
第 1 条	格式锁定：只输出 JSON 数组，不要任何解释文字/前后缀/代码块	最常见故障：模型"礼貌地"在 JSON 外加一句"好的，以下是您的计划："
第 2 条	完整字段示例：给出一个填好真实值的对象	字段名拼写漂移（due_date 写成 deadline）、字段缺失；示例即 Schema
第 3 条	引用完整性：dependencies 只能引用已出现的 task_id	模型编造不存在的依赖 ID，后端无法映射父子任务
第 4 条	枚举白名单：priority 只能是「高/中/低」	模型自创"紧急""P0"等无法入库的值
第 5 条	类型声明：estimated_hours 为数字	"约两周"这类自然语言值
第 6 条	数量边界：6~12 个	模型要么只给 2 个任务，要么给 50 个
第 7 条	日期边界：必须在今天~截止日之间，且不倒挂	排期超出项目截止日（校验层会拒绝，浪费重试 token）
第 8 条	数值自洽：总工时 ≈ 每周小时 × 周数 × 0.85（动态计算后填入）	计划工作量远超用户实际可用时间；0.85 给突发情况留缓冲
第 9 条	领域约束：跳过已完成工作、从当前阶段排起	重复安排已做过的事

条目	手法	防的是什么
第 10 条	结构约束：形成依赖推进链路	一堆互不相关的任务，看板失去意义

实测效果：9 个任务、严格 JSON、**总耗时 102 小时 = 10 小时 × 12 周 × 0.85，与约束完全一致**、依赖链 T1→T9 完整、日期全部落在截止日内。

3.3.2 动态重规划 Prompt 解析

重规划的难点：模型的输出要**引用数据库里真实存在的任务**，改动要**能程序化地应用到任务表**。Prompt（`backend/app/services/replan_service.py`）：

```
你是科研项目管理专家，负责对延期的科研任务计划做重规划。
用户会提供项目信息与全部任务的当前状态（含本地规则引擎计算出的风险与原因）。
你的任务：只对需要调整的任务给出调整建议，输出严格 JSON（不要任何解释文字）：
{"plan_content": "给用户看的整体调整说明",
 "impact_summary": "对项目整体进度的影响评估",
 "changes": [{"task_id": 数字, "action": "reschedule 或 reprioritize",
              "title": "任务标题", "planned_start_date": "YYYY-MM-DD",
              "planned_end_date": "YYYY-MM-DD", "priority": "high/medium/low",
              "estimated_hours": 数字, "reason": "这条调整的理由"}]}
约束：
1. task_id 必须是输入数据中已有的任务 id，不要编造新任务；
2. action 只能是 reschedule（改起止日期/工时）或 reprioritize（改优先级）；
3. 所有日期必须在今天到项目截止日期之间，planned_end_date 不得早于
planned_start_date；
4. 优先调整有风险的任务，正常任务尽量不改；
5. 阻塞类任务优先建议先解决前置任务，必要时顺延后续任务日期；
6. reason 用中文写清楚为什么这样调；
7. 如果没有任务需要调整，changes 为空数组，plan_content 说明当前计划健康。
```

与任务生成 Prompt 相比的三个进阶设计：**上下文由代码打包**（后端先运行本地风险引擎，把每个任务的 task_id/status/progress/优先级/起止日期/工时/风险等级/风险原因组装成 JSON 快照随 Prompt 发出，模型只负责"给出调整建议"，基础计算不交给它——这本身就是需求护栏）；**输出可执行性约束**（task_id 必须真实存在、action 限定枚举、日期限定区间，保证 adopt 接口可逐条程序化应用）；**"健康计划"分支**（第 7 条允许 changes 为空数组，避免无风险时强行制造调整）。

实测效果：真实 DeepSeek 给出 3 条调整全部引用真实任务 id，还主动识别出"总工时需求约 58 小时，超出剩余可用工时约 18 小时"的缺口并在 impact_summary 中建议提高投入。

3.3.3 文献问答 Prompt：用 Prompt 对抗幻觉

文献问答的诉求相反——要"**只依据给定片段**"的自然语言回答，但要带**可校验的结构化引用**。Prompt（`backend/app/services/document_service.py`）：

```
你是严谨的文献阅读助手。用户会提供若干文献片段和一个问题。
回答规则（必须严格遵守）：
1. 只能依据提供的文献片段回答，禁止使用你的外部知识；
2. 如果片段中找不到答案，answer 中明确写「文献片段中未提及该内容」；
```

```
3. 引用必须来自片段：citations 中的 chunk_index 只能是片段编号，
   quote 必须逐字摘自对应片段原文（不超过 60 字）；
4. 只输出一个 JSON 对象，不要任何解释文字或 Markdown 代码块，格式：
   {"answer": "回答正文", "citations": [{"chunk_index": 1, "quote": "原文原句"}],
"confidence": 0.85}
5. confidence 为 0~1 的数字，表示答案的可信程度。
```

三层反幻觉设计：**规则 1** 切断外部知识来源；**规则 2** 给"不知道"一个合法的输出形式（模型不会为了有话说而编造）；**规则 3 + 代码校验**要求 quote 逐字摘自片段且 chunk_index 必须落在真实片段编号内——校验层会丢弃任何引用不存在片段的 citation。实测：置信度 0.95，3 条引用均逐字摘自原文。

3.3.4 汇报/周报类 Prompt：结构锁 + 反编造

汇报类输出是 Markdown 而非 JSON，约束重心转向**章节结构**和**数据忠实性**：

场景	关键约束原文
汇报中心	"必须包含以下五个章节，使用二级标题：## 本周完成、## 未完成及原因、## 当前风险、## 下周计划、## 需要导师支持"；"**严格依据给定数据撰写，不得编造数据中没有的内容**；数据不足的章节写「无」"；风格随类型切换（meeting=要点式口语 / supervisor=正式书面语）
周报	"内容包括：本周工作概述、主要进展（按项目）、投入时间统计、下周建议。语气客观、积极，篇幅 300~500 字。直接输出周报正文，不要输出与周报无关的解释。"

汇报 Prompt 的输入同样是**代码聚合后的结构化 JSON**（完成任务/未完成及原因/风险/进度记录/重规划历史，按时间窗口过滤），模型只做"数据 → 行文"的转换。实测：AI 从进度备注"数据格式问题卡住"中推断出"需导师协调数据来源"——结构化数据喂得准，模型才能写出有价值的推断。

3.3.5 共同技巧汇总

技巧	具体做法	出现场景
人设锚定	"你是资深科研项目规划专家 / 严谨的文献阅读助手 / 研究生的科研助理"	全部
格式锁定	"只输出 JSON 数组，不要输出任何解释文字、前后缀、Markdown 代码块标记"	全部 JSON 场景
示例即 Schema	给出一个填好真实值的完整对象示例	任务生成、重规划、文献问答
枚举白名单	priority ∈ {高/中/低}、action ∈ {reschedule, reprioritize}	任务生成、重规划
引用完整性	dependencies 只能引用已出现的 task_id；chunk_index 只能是给定片段编号	任务生成、文献问答
数值边界	6~12 个任务、日期区间、总工时公式、confidence ∈ [0,1]	任务生成、文献问答
领域约束	跳过已完成工作、阻塞任务先解前置	任务生成、重规划

技巧	具体做法	出现场景
反编造条款	"不得编造数据中没有的内容""片段中找不到就写未提及"	汇报、文献问答
低温采样	`temperature: 0.3`（拆解求稳定，配置于 ai_client）	全部
输出分支合法化	"changes 为空数组 = 计划健康"、"数据不足的章节写「无」"	重规划、汇报

3.4 代码层防线：Prompt 之外的四道保险

Prompt 只是第一道防线。代码层实现了四道防线，把"模型不听话"从故障降级为可恢复事件：

```
DeepSeek 原始输出
  → 防线一 容错解析（剥围栏 → 整体解析 → 截取首 [ 到末 ] 兜底；对象/数组双兼容）
  → 防线二 逐字段校验与归一化（硬性约束失败抛错，软性偏差自动修正）
  → 防线三 自动重试（失败时把"纠错说明"追加进对话，最多重试 2 次）
  → 防线四 友好降级（仍失败 → 502 中文提示；无 Key → 演示模式）
```

防线一：容错解析。模型输出的三种"不听话"形态都被兼容（`_extract_json`）：Markdown 围栏包裹（正则剥掉 ```` ```json ```` 围栏）、JSON 前后有解释文字（截取第一个 `[` 到最后一个 `]`）、把数组包成 `{"tasks": [...]}` 对象（dict 时取 tasks/plan 字段）。

防线二：逐字段校验与归一化。解析成功不等于可用，硬性约束失败抛错触发重试，软性偏差自动归一化：

校验项	策略	类型
title 缺失 / due 早于 start / due 晚于项目截止日	抛错（触发重试）	硬性
task_id 缺失/重复	缺则补 T1/T2…，重复则加后缀 `-2`	归一化
priority 非法（含英文别名）	归一为「中」	归一化
estimated_hours 非法/缺失	归一为 8 小时，裁剪到 [0.5, 1000]	归一化
dependencies/changes 引用不存在的对象	忽略该引用/丢弃该条调整	归一化
存在风险任务但 changes 无任何可应用调整	视为格式问题抛错重试（模型"敷衍"的典型形态）	硬性
citations 的 chunk_index 不在给定片段内	丢弃（不得编造）	归一化
任务数超过上限	截断到 30 个（防御异常输出）	归一化

防线三：自动重试（带纠错说明）。重试不是原样重发：失败时把具体原因（如"任务「x」的 due_date 晚于项目截止日期"）作为纠错说明追加进对话——第二次请求里模型知道自己错在哪。共 3 次尝试（`1 + AI_FORMAT_RETRY_TIMES`，RETRY_TIMES=2），耗尽则抛 `DeepSeekError`。

防线四：友好降级。重试耗尽 → 502 + 中文提示（附"可删除 Key 使用演示模式"的出路）；网络/超时/401/402/429/5xx → `ai_client.py` 统一映射中文提示（"API Key 无效或已过期，请检查 backend/.env""账户余额不足，请充值后重试"等）；未配置 Key → **演示模式**（预设样例计划/本地规则模拟重规划/检索片段直出/模板汇报），全部页面明确标注"演示模式"徽章。

3.5 意图控制的效果验证

验证点	数据
任务生成与约束自洽	9 个任务，总工时 102h = 10h×12 周×0.85（第 8 条约束精确命中）
依赖链完整性	T1→T9 单链，确认写入时 8 个父子映射全部正确
重规划输出可执行	3 条调整全部引用真实任务 id，adopt 接口逐条应用成功（applied=3）
反幻觉	文献问答置信度 0.95，3 条引用逐字摘自原文；汇报"需要导师支持"从备注推断而非编造
格式容错	围栏/对象包裹/垃圾文本/超期任务 4 类畸形输出全部正确处理
错误映射	无效 Key → 502 中文提示；超时/402/429/5xx 全部分支有中文文案
全链路降级	无 Key 时 4 个 AI 功能全部可走通演示模式，核心功能不依赖 AI

3.6 可复用的意图控制清单

约束写代码的 Claude：需求含输入定义、输出结构（字段级）、编号行为清单、验收标准与测试素材；护栏条款（不改已有代码/不换技术栈/先扫描项目结构）；大功能先要文件清单与接口设计，确认后再动工；要求交付测试结果与浏览器操作步骤；降级路径（无 Key/无网络）写进需求；验收量化（冒烟测试 N/N、build 通过、零改动检查）。

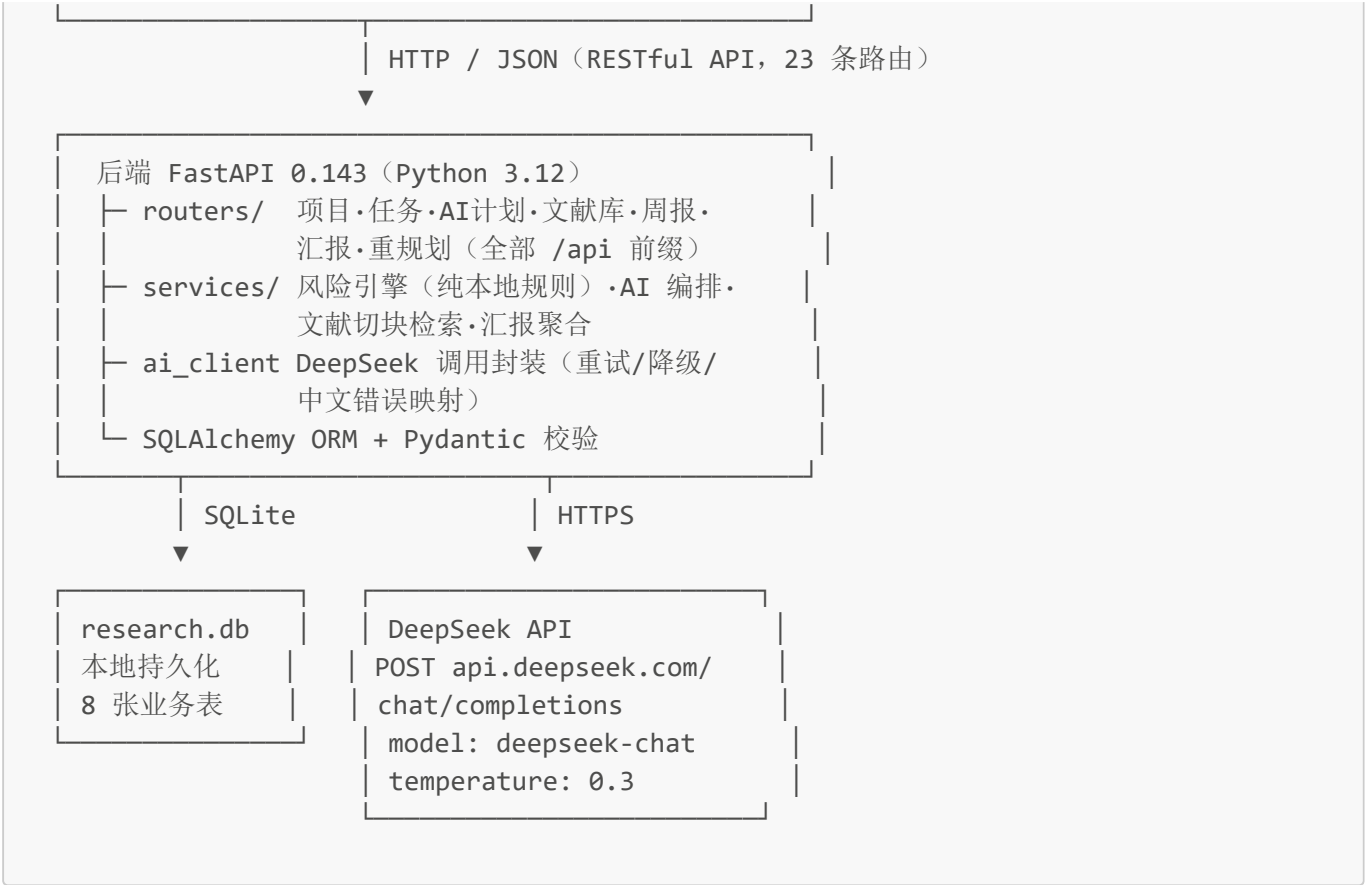
约束出内容的 DeepSeek：人设 + 格式锁定（"只输出 JSON，不要解释文字"）；完整字段示例（示例即 Schema）；枚举白名单、数值边界、日期边界；引用完整性（只能引用输入中已有的 ID/编号）；反编造条款 + "不知道"的合法输出形式；temperature 调低（0.3）；代码层四道防线（容错解析 → 逐字段校验归一化 → 带纠错说明的重试 → 友好降级）。

---

4. 系统架构

4.1 总体架构

```
┌──────────────────────────────────────────────────┐
│  浏览器（React 19 + TypeScript + Vite 7）          │
│  Tailwind CSS + shadcn/ui + Recharts             │
│  axios 统一封装（同源 /api，Vite 代理转发）          │
```



4.2 前端架构

层	内容
页面	首页仪表盘、新建项目向导、任务看板、项目仪表盘（4 个 Tab：任务看板/文献库/汇报中心/重规划）、周报生成
组件	看板三列、任务表单、文献库（上传/问答/笔记）、汇报中心、重规划（风险列表/新旧对比表/历史）、仪表盘图表、shadcn/ui 组件库（含自研 native-select 通用下拉）
基础设施	`services/api.ts`（axios 统一封装，错误统一 toast）、`types/index.ts`（前后端共享类型）、`useAsyncData`（加载三态）、`useTheme`（深浅主题）
图表	Recharts（完成趋势折线、状态分布环图、投入统计）

4.3 后端架构（分层）

层	文件	职责
入口	`main.py`	CORS、路由注册、异常处理（422 中文提示、GBK 编码防护）
配置	`config.py`	数据库路径、DeepSeek 配置（读 .env）、风险阈值、重试次数
路由层	`routers/`（8 个模块）	HTTP 层：参数校验、错误码转换（502/422/404/400），不写业务逻辑
服务层	`services/`（7 个模块）	业务编排：risk（风险引擎）、ai_planner、replan_service、document_service、meeting_report、report、serializers

层	文件	职责
AI 客户端	`ai_client.py`	DeepSeek HTTP 调用封装：超时/断网/401/402/429/5xx → 中文错误映射
数据层	`models.py` + `database.py`	SQLAlchemy 2.0 ORM，8 张业务表，外键级联删除

4.4 数据库设计（SQLite，8 张业务表）

```
project 1—N task（task 自关联 parent_id = 前置/里程碑）
project 1—N progress_record（进度历史，只增不删，供图表）
project 1—N replan_history（重规划留档：原计划快照/新计划/状态）
project 1—N document 1—N doc_chunk（文献 + 500~800 字切片）
project 1—N doc_qa（问答，citations_json/confidence/is_saved 笔记标记）
project 1—N reports（汇报，Markdown content，可编辑导出）
```

表	关键字段	说明
project	name, goal, deadline, weekly_hours, status	科研项目主体，状态 active/completed/paused
task	project_id, parent_id, title, status(todo/doing/done), priority(high/medium/low), planned_start/end_date, estimated_hours, progress, is_milestone, sort_order	两级结构：parent_id 表示前置依赖或所属里程碑
progress_record	task_id, progress, hours_spent, note, created_at	每次进度更新追加一条，历史只增不删
replan_history	project_id, trigger_type, risk_tasks_json, plan_content, changes_json, status(pending/adopted/rejected), decided_at	changes_json 内含原计划快照与新计划（对比表的数据来源）
document	project_id, title, file_name, full_text, chunk_count	上传 PDF 提取的全文
doc_chunk	document_id, chunk_index, content	检索最小单元，每段约 500~800 字
doc_qa	project_id, document_id, question, answer, citations_json, confidence, source, is_saved	问答记录，is_saved 即笔记
reports	project_id, title, report_type(meeting/supervisor), week_start, week_end, content, source	汇报留档，content 为 Markdown

全部外键 `ON DELETE CASCADE`：删除项目即级联清理其任务、进度、重规划历史、文献、问答与汇报。

4.5 DeepSeek API 集成

调用点唯一：`ai_client.DeepSeekClient.chat(messages)`；配置集中在 `config.py`（读 `backend/.env`）；请求参数 `model=deepseek-chat`、`temperature=0.3`（结构化输出求稳定）、`timeout=60s` 可配置。

异常	HTTP 返回	用户看到的提示
超时	502	DeepSeek 响应超时（60 秒），请稍后重试（可在 backend/.env 调大 DEEPSEEK_TIMEOUT）
网络错误	502	无法连接 DeepSeek API，请检查网络是否可访问 api.deepseek.com
401	502	DeepSeek API Key 无效或已过期，请检查 backend/.env 中的 DEEPSEEK_API_KEY
402	502	DeepSeek 账户余额不足，请充值后重试
429	502	DeepSeek 请求被限流，请稍后重试
5xx	502	DeepSeek 服务暂时不可用（HTTP 5xx），请稍后重试
格式多次校验失败	502	AI 返回的计划格式多次校验失败，请稍后重试；也可以删除 .env 中的密钥使用演示模式
未配置 Key	200（演示模式）	页面出现"演示模式"徽章 + 说明文案

4.6 关键设计决策

1. **风险计算不依赖大模型**：逾期/阻塞/落后/工时不足全部本地规则实时计算，DeepSeek 只负责给出调整建议。可解释、零 token 成本、断网可用。
2. **API Key 只在后端**：存于 `backend/.env`（已被 .gitignore 排除），前端代码与接口响应中绝不出现。
3. **AI 是增强不是必需**：未配置 Key 时全部功能进入演示模式，核心 CRUD/风险/图表照常可用。
4. **数据持久化**：SQLite 落盘 `backend/research.db`，重启服务、刷新网页数据不丢。

---

5. 核心功能：任务生成、风险计算、重规划逻辑

5.1 AI 科研计划生成

```
用户提交（主题/截止日/每周小时/当前阶段/已完成工作）
  → Pydantic 校验（422 中文提示）
  → 无 Key? → 演示模式样例计划（6 个任务按通用科研路径排布）
  → 有 Key → 组装强约束 Prompt（3.3.1 节）→ DeepSeek
  → 容错解析 → 逐字段校验归一化 → 失败自动重试 2 次（附纠错说明）
  → 返回预览（不写库）→ 用户确认 → POST /api/ai/plan/confirm：
     项目+任务原子写入（服务端重新校验，不信任前端回传；依赖映射为 parent_id）
```

关键实现：总工时预算动态注入 Prompt（`每周小时 × 周数 × 0.85`），模型排期不会超出用户实际可用时间；确认写入时任务描述合并"description + 预期产出 + 安排理由"，理由不丢失。

5.2 本地风险引擎（纯规则，零 AI）

`services/risk.py` 实现 4 条任务级规则 + 项目级聚合，每条风险输出**人话原因**：

#	规则	条件	等级	示例输出
1	逾期	未完成且 planned_end_date < 今天	高	"已逾期 2 天，进度仅 40%"
2	前置阻塞	父任务（前置）未完成	中	"前置任务「核心实验」尚未完成，当前任务被阻塞"
3	进度落后	计划时间已过 >30% 且实际进度落后应达进度 >20 个百分点	中	"进度落后计划 25%，当前进度 40%"
4	剩余工时不足	剩余工作量按每周可用小时折算后超出剩余天数容量	中	"剩余约 20 小时工作量，距计划截止还有 10 天（按每周 10 小时约可用 14 小时），可能不足"

项目级聚合：任一任务高风险 → 项目高风险；≥3 个任务中风险 → 项目中风险；项目整体进度落后 >20 个百分点 → 中风险；项目本身逾期 → 高风险。

关键实现细节：风险等级比较使用显式排序表 `{none:0, medium:1, high:2}`——开发中曾踩过 `max()` 对字符串按字典序比较的坑（`"medium" > "high"`），导致逾期任务被算成中风险；风险随项目列表/详情响应附带返回（`task.risk` + `task.riskReason`），零额外请求；阈值集中在 `config.py` 便于调整。

5.3 DeepSeek 动态重规划（核心创新）

```
用户点【重新规划】→ 后端构建任务快照（每个任务状态 + 本地风险引擎结果）
  → 组装 Prompt（项目信息 + 剩余天数 + 每周工时 + 快照 JSON，见 3.3.2）
  → DeepSeek 输出 {plan_content, impact_summary, changes[]}
  → 解析校验（task_id 必须真实存在、action 枚举、日期收敛）
  → 生成 ReplanHistory 记录（status=pending，含原计划快照 + 新计划）
  → 前端展示：方案说明 + 影响评估 + 原计划 vs 新计划对比表（改动行高亮）
  → 【接受调整】→ 逐条应用 changes → status=adopted；【放弃】→ 任务表不变 → rejected
  → 历史记录永久留档（时间、来源 deepseek/demo、状态）
```

关键实现：**采纳时的安全收敛**——应用每条调整后再校验一次，planned_end_date 早于开始日则取开始日、晚于项目截止日则收敛到截止日，即使模型输出有瑕疵，落库数据依然合法；**演示模式重规划**（无 Key）——本地规则模拟（按"剩余工时 ÷ 每周小时"折算顺延天数，逾期任务优先级提为 high）；**历史留档**——changes_json 同时保存 original_tasks 快照与 changes，对比表和历史查看都从这份档案读取。

实测样例（真实 DeepSeek 输出，测试项目"核心实验已逾期 2 天、论文撰写被阻塞、答辩准备正常"）：

任务	调整	新日期	优先级	理由（节选）
核心实验（已逾期）	reschedule	10-12 ~ 10-31	高	已逾期且仅完成 40%，顺延以留出足够时间完成剩余 60% 工作量
论文撰写（被阻塞）	reschedule	11-01 ~ 11-08	中	被前置任务阻塞，需待核心实验 10-31 完成后再启动

任务	调整	新日期	优先级	理由（节选）
答辩准备（正常）	reschedule	11-08 ~ 11-11	中	原计划与顺延后的论文撰写时间冲突，压缩调整

影响评估："总工时需求约 58 小时，超出剩余可用工时约 18 小时，存在工时不足风险，建议提高每周投入或压缩论文撰写范围。"

5.4 其他核心功能

功能	实现要点
文献库	pypdf 提取文字；扫描版检测（每页提取字数 < 30 判定，返回 400 明确提示）；按段落 500~800 字切块；jieba 分词（含停用词表）+ 词频×词长打分取 top5 片段；问答见 3.3.3；答案可保存为笔记；点击引用自动滚动定位原文并高亮
汇报中心	按时间窗口聚合：范围内完成任务/未完成及原因/当前风险/下周任务/重规划历史 → JSON 进 Prompt → Markdown 五章节；编辑保存、复制、导出 .md
周报	近 7 天活动聚合（纯本地统计）→ DeepSeek 生成 300~500 字周报；无 Key 模板拼接
看板与图表	三列看板（待开始/进行中/已完成）+ 风险徽章；进度记录驱动趋势图；仪表盘统计卡片

---

6. AI 协同开发演进：3 组完整 Prompt 案例

> 完整过程记录存于 `docs/prompt-log/`（01-arch-design.md、02-deepseek-task-generate.md、03-risk-replan+repair-bug.md），本章呈现 3 组有代表性的完整 Prompt 及其执行结果。

6.1 案例一：产品需求与架构设计（M1）

项目从零起步。这组对话产出了 `docs/product-requirements.md`（PRD）与系统架构设计，定稿 MVP 范围（第 2.5 节 6 项决策）。核心成果：5 大痛点 → 5 项解法对齐表（第 1.2 节原型）；架构与数据流及三条关键设计原则（前端不直接调 DeepSeek、风险识别纯本地规则、AI 失败自动降级）；4 张核心表（project/task/progress_record/replan_history）；18 个 API 接口规划（"风险不单独设接口、随响应附带"在此定稿）；8 个里程碑开发顺序（每阶段独立可演示、有验收标准）；风险预案——**"DeepSeek 输出格式不稳定 → 强约束 Prompt + 校验 + 重试 + 降级"（第 3 章策略的源头）**。特征：以"方案 + 6 项待确认决策"收口，把产品决策权交还用户，避免 AI 自作主张定范围。

6.2 案例二：DeepSeek 任务规划模块（M4~M6 的核心）

用户原始 Prompt（完整）：

```
现在开发DeepSeek API任务规划模块，后端目录./backend。
功能描述：
用户输入：研究主题、项目截止日期、每周可用小时、当前研究阶段、已完成工作。
```

```
后端组装Prompt发给DeepSeek API（deepseek-chat），强制模型返回严格JSON格式，不要自然语言文字。
JSON结构定义：
[
  {
    "task_id": "字符串",
    "title": "任务标题",
    "description": "任务描述",
    "start_date": "YYYY-MM-DD",
    "due_date": "YYYY-MM-DD",
    "priority": "高/中/低",
    "expected_deliverable": "预期产出",
    "dependencies": ["前置task_id"],
    "estimated_hours": 数字,
    "rationale": "为什么安排这个任务"
  }
]
需求：
1. 从.env读取DeepSeek密钥，密钥严禁暴露到前端。
2. 增加JSON校验：DeepSeek返回结果如果格式错误，自动重试2次；重试失败返回友好提示。
3. 前端页面：用户输入科研信息，点击【生成科研计划】，页面显示加载动画；拿到计划后预览，
   用户点击确认，任务才存入SQLite数据库。
4. 增加演示模式：当没有DeepSeek密钥时，直接返回预设样例任务，保证项目可以演示运行。
5. 捕获网络超时、API调用失败、余额不足等异常，页面友好提示。
6. 不要改动已经写好的任务CRUD代码。

完成后给我测试案例，测试文本：
"研究主题：虚拟世界智能体行为规划，12周完成论文初稿，每周投入10小时，当前处于文献调研阶段"
```

执行过程要点：

1. 环境预检先行：确认 .env 已配 Key、httpx 未装——**先装依赖再写代码**（用户 8000 后端带 --reload，新代码 import httpx 会直接崩）。
2. 按 PRD 模块划分落地（config → ai_client → services/ai_planner → routers/ai），**原有任务 CRUD 一行未动**；生成预览不写库、确认后原子写入。
3. 三实例并行测试：8001 无 Key（演示模式）、8002 真实 Key、8003 无效 Key。
4. **真实 Key 测试暴露配置 bug**：请求 404，curl 探测两端点确认 Key 有效，定位为 `.env` 的 `DEEPSEEK_BASE_URL` 填了完整端点，客户端再拼接造成 `.../chat/completions/chat/completions`。修复 .env 并让代码兼容两种写法。
5. 真实生成结果：9 个任务、严格 JSON、**总耗时 102h 精确等于 10×12×0.85**、依赖链完整、日期全部落在截止日内；解析/校验单元测试覆盖 5 类畸形输出全部通过；回归冒烟 36/36。

交付后追加两轮修复（同一日志内，展示"意图反馈闭环"）：用户反馈"当前研究阶段下拉弹不出选项" → 定位为 Base UI Select 在 `value={null}` 状态下的已知毛病 → 换原生 select；用户反馈"看板无法选择项目" → 确认 Select 整体性故障 → 自研 `native-select.tsx` **一次性替换全部 4 处下拉**。

特征：完整规格（JSON 结构直接写进需求）+ 护栏（第 6 条）+ 强制测试（测试文本由用户指定）+ 真实环境验证。

6.3 案例三：风险预警 + 动态重规划 + 修复后端无法加载（M7，核心创新）

用户原始 Prompt（完整）：

```
我的开发核心创新功能：科研进度风险预警 + DeepSeek动态重规划，后端./backend。
流程：
1. 后端本地计算风险（不依赖大模型）：判断任务是否逾期、剩余工时是否不足、
   前置任务未完成阻塞后续任务。输出风险原因，可解释。
2. 用户点击【重新规划】，后端把当前项目全部任务状态、剩余时间、每周可用工时打包，
   作为上下文发给DeepSeek API。
3. DeepSeek输出新的任务计划JSON，附带调整理由、对项目整体进度的影响。
4. 前端展示：原计划 vs 新计划对比表格，高亮改动内容。
5. 用户点击【接受调整】，数据库更新任务，同时保存本次重规划记录
   （原计划、新计划、时间、理由），支持查看历史重规划记录。
6. 同样支持演示模式，无DeepSeek密钥时使用模拟重规划结果。
7. 风险判断逻辑写在后端代码，不要交给大模型做基础计算；大模型只负责给出调整建议。
8. 不改动已有任务生成和CRUD代码。

完成后给我测试流程：创建项目，标记部分任务延期，触发风险预警，执行重规划，
查看新旧对比和历史记录。
```

执行过程要点：

1. 扩展风险引擎（新增"前置阻塞""剩余工时不足"两条规则，纯本地计算），实现 replan_service（快照组装 → 强约束 Prompt → 解析校验 → 重试 → 演示模式规则模拟）。
2. 构造风险场景实测：逾期任务 risk=high "已逾期 2 天，进度仅 40%"；被阻塞任务 risk=medium；正常任务 none；项目级 high——风险引擎全对。
3. **测试当场抓住两个 bug（"强制测试"的价值）**：演示模式重规划 0 条调整（风险类型名与中文原因列表混用，筛选条件写反）；逾期任务被算成 medium（`max()` 字符串按字典序比较，`"medium" > "high"`）——修复为显式等级排序表。
4. 真实 DeepSeek 重规划：3 条调整全部引用真实任务 id，影响评估算出 18 小时工时缺口（第 5.3 节样例）。
5. 采纳（applied=3，任务表实际更新）、放弃（任务表不变）、历史留档全链路通过；回归冒烟 36/36、前端构建通过、用户 8000 实例自动加载 4 条新路由。

用户追加反馈与修复（后端无法加载）：用户报 `ModuleNotFoundError: No module named 'httpx'`，所有界面加载失败。定位：报错堆栈显示激活的是**仓库根目录的 `.venv`**（只有基础包），完整依赖装在 `backend\venv`——**不是代码问题，是虚拟环境用错**。修复：把完整依赖装进根目录 .venv，实测启动成功、23 条路由注册、36/36 冒烟通过。

特征：职责边界护栏（第 7 条：基础计算不给大模型）+ 完整流程定义（1~5 步即需求本身）+ 验收流程由用户指定；并展示了"测试驱动下 bug 当场现形"的完整闭环。

6.4 三组案例的对比：意图约束手法的演化

维度	案例一（架构设计）	案例二（AI 任务规划）	案例三（风险+重规划）
需求形态	开放式讨论，以方案+决策收口	完整规格：JSON 结构+7 条需求+测试文本	完整流程定义（1~5 步）+ 8 条需求
护栏强度	无（从零开始）	"不改已有 CRUD"	"不改已有代码"+"基础计算不给大模型"
验收方式	用户审阅方案	用户指定测试文本，三环境实测	用户指定验收流程，构造风险场景
测试暴露的问题	—	BASE_URL 配置 404	2 个逻辑 bug + venv 环境问题
用户反馈闭环	6 项 MVP 决策确认	下拉框 2 轮修复	后端无法加载修复

规律：**需求越结构化（规格+护栏+验收），一次通过率越高；但真实环境测试依然不可替代**——三组案例中所有"意外"都来自模型和代码之外的因素（配置、组件库兼容性、虚拟环境、字典序），只有"用真实数据跑一遍"才能暴露。

---

7. 测试结果

7.1 测试体系

层	手段	内容
后端回归	`backend/smoke_test.py`	36 项接口冒烟测试（CRUD + 校验 + 错误处理 + 持久化），一键执行
AI 功能验证	三实例实测	无 Key（演示）/ 真实 Key / 无效 Key 三个后端实例分别验证
解析校验	单元断言	`_extract_json` / `validate_and_normalize` 对 5 类畸形输出的处理
前端	`npm run build`	tsc 类型检查 + Vite 构建
全链路	脚本走查	上传 PDF → 提问 → 引用跳转；延期 → 风险 → 重规划 → 采纳 → 历史

7.2 冒烟测试（36/36 通过）

`smoke_test.py` 仅操作名称带「【冒烟测试】」前缀的数据，不污染用户数据。覆盖：健康检查；项目 CRUD + 校验（空名称/0 小时/过去截止日/缺字段 → 422 中文提示；**GBK 编码请求体 → 400/422 而非 500**——Windows 终端中文防护；404）；任务 CRUD + 校验；进度记录（新增/历史/进度同步）；持久化（重新查询数据仍在）；删除级联（删项目 → 任务/进度/重规划历史全部清理）。**结果：通过 36 项，失败 0 项**（在 demo/真实 Key/修复后实例上多次执行均如此）。

7.3 AI 功能实测结果（真实 DeepSeek）

测试	结果
科研计划生成（虚拟世界智能体行为规划，12 周，每周 10 小时）	9 个任务；严格 JSON；总耗时 102h = 10×12×0.85（与约束一致）；依赖链 T1→T9 完整；日期全部落在截止日内
确认写入 → 持久化	项目 + 9 任务落库；依赖正确映射为父子任务（8 个 parent_id）；重新查询数据仍在
动态重规划（构造逾期+阻塞场景）	3 条调整全部引用真实任务 id；影响评估识别 18 小时工时缺口并给出建议
采纳/放弃	采纳 applied=3，任务表日期/优先级实际更新；放弃后任务表不变；历史记录 5 条含原计划/新计划/时间/状态/来源
文献问答（上传自制中文论文 PDF，问"研究方法是什么"）	置信度 0.95；准确答出三种研究方法；3 条引用逐字摘自原文
扫描版 PDF	400 "该 PDF 可能是扫描版（没有文字层），暂不支持"
中文 PDF 切块	提取 1027 字 → 切 2 段（767 + 259 字，符合 500~800 规则）
组会周报 / 导师汇报	五章节齐全，内容全部来自真实任务数据；"需要导师支持"从备注推断"协调数据来源"

7.4 错误处理与降级测试

测试	结果
无效 Key	502 "DeepSeek API Key 无效或已过期，请检查 backend/.env 中的 DEEPSEEK_API_KEY"
空主题 / 过去截止日	422 中文提示
解析/校验单元测试	围栏包裹/对象包裹/垃圾文本/超期任务/日期倒挂全部正确处理
演示模式（无 Key）	计划生成 6 个样例任务（依赖链 T1→T6）；重规划本地规则模拟；文献问答返回检索片段；周报/汇报模板生成——全部带"演示模式"标记
演示模式重规划	逾期任务顺延至截止日内 + 优先级提为 high
前端构建	`npm run build` 通过（含 tsc 类型检查）

7.5 测试中发现并修复的问题清单（测试的价值证明）

#	问题	定位	修复
1	真实 Key 请求 404	`.env` 的 BASE_URL 填了完整端点，拼接出 `.../chat/completions/chat/completions`	改 .env 为 base 地址；代码兼容两种写法
2	周报聚合 500	列表推导循环变量笔误 `t` vs `r`（UnboundLocalError）	修正变量

#	问题	定位	修复
3	演示模式重规划 0 条调整	风险类型名与中文原因混用，筛选条件写反	统一用类型名判断
4	逾期任务风险等级被算成 medium	`max()` 字符串按字典序比较，`"medium" > "high"`	显式等级排序表
5	后端无法加载，前端全挂	根目录 .venv 缺 httpx 等依赖（虚拟环境用错）	补装完整依赖并实测 36/36
6	下拉框弹不出选项	Base UI Select 在 `value=null` 及整体性故障	自研原生 select 组件替换全部 4 处
7	用户 8000 进程卡死	密集文件修改触发 --reload 连续重载挂住	结束进程，提示重启

8. 可复现性说明

8.1 代码仓库与演进历史

提交	时间	里程碑	内容
361d6eb	2026-10-10 21:58	M1 架构设计	PRD + 系统架构 + 4 张核心表结构设计定稿
8a82885	2026-10-10 22:15	M2 前端静态 UI	全部页面、看板、图表 UI，使用模拟静态数据
fcc9c7b	2026-10-12 15:02	M3 后端数据层	FastAPI + SQLite 全量 CRUD、入参校验、风险引擎 v1、前端接入真实接口
40b9db6	2026-10-12 18:17	M4~M6 AI 能力	DeepSeek 客户端、计划生成/预览/确认、演示模式、周报生成、文献库、汇报中心
0cfc1eb	2026-10-12 18:46	M7 核心创新	风险预警增强（前置阻塞/工时不足）+ DeepSeek 动态重规划 + 新旧对比 + 历史留档 + 修复后端无法加载

8.2 环境要求与依赖

Python 3.10+（实测 3.12.3）、Node.js 20+（实测 v20.18.0）、npm 10+；Windows/macOS/Linux 均可。后端依赖（`backend/requirements.txt`）：fastapi 0.143、uvicorn、sqlalchemy 2.x、pydantic 2.x、httpx、python-dotenv、pypdf、jieba、python-multipart。

8.3 启动步骤（10 分钟复现）

```
cd E:\ResearchPilot\backend
python -m venv .venv                # 首次
.venv\Scripts\activate              # 每次启动前
pip install -r requirements.txt     # 首次
copy .env.example .env              # 首次；填入真实 DeepSeek Key（不配也能跑演示模式）
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

看到 `Application startup complete` 即成功；浏览器打开 http://127.0.0.1:8000/docs 可查看全部接口分组（AI/周报/文献库/汇报中心/重规划等 23 条路由）。

```
cd E:\ResearchPilot\frontend
npm install     # 首次
npm run dev     # 终端打印访问地址（如 http://localhost:5173）
```

浏览器打开（Ctrl+F5 强制刷新）。建议体验路径：新建项目 → AI 生成科研计划 → 看板流转任务 → 故意设置一个过期任务 → 重规划 Tab 看风险预警与新旧计划对比 → 汇报中心生成导师汇报。

8.4 配置与演示模式

```
DEEPSEEK_API_KEY=sk-xxx          # 留空 = 演示模式
DEEPSEEK_BASE_URL=https://api.deepseek.com   # 填 base 地址，不要带
/chat/completions 后缀
DEEPSEEK_MODEL=deepseek-chat
DEEPSEEK_TIMEOUT=60
```

`.env` 已被 .gitignore 排除，`.env.example` 为模板。**演示模式**：不配置 Key（或留空）时后端自动进入演示模式，全部页面显示「演示模式」徽章——AI 计划生成返回预设样例任务（6 个带依赖链）、重规划用本地规则模拟、文献问答返回关键词检索片段、汇报/周报用真实数据拼模板；CRUD、风险预警、看板、图表不依赖 AI，完全照常可用。**一键回归**：后端启动后执行 `cd backend && .venv\Scripts\python smoke_test.py`，36 项全过即接口全部正常。

8.5 常见问题（FAQ 摘录，完整版见 README）

现象	原因与解决
前端提示「无法连接后端服务」	后端没启动或 8000 被旧进程占用
前端打开了但数据是旧的	Vite 旧实例占端口，关掉旧终端重启，浏览器 Ctrl+F5
`ModuleNotFoundError: No module named 'httpx'`	激活的虚拟环境不是装过依赖的那个（本项目真实发生过，见案例三）
DeepSeek 返回 404	BASE_URL 必须填 `https://api.deepseek.com`（不带 /chat/completions 后缀）

现象	原因与解决
401 / 402 提示	401 检查 Key；402 充值余额。均为前端 toast 中文提示，不会崩溃
上传扫描版 PDF 提示不支持	扫描版无文字层，需含文字层的 PDF（如知网/arXiv 原文）

9. 不足与未来优化方向

9.1 当前不足（如实评估）

#	不足	影响与说明
1	无登录与多用户	仅单机单人使用（MVP 决策）；数据表无用户维度
2	文献检索是关键词级	jieba 词频打分只命中字面匹配，同义改写（"研究方法" vs "实验设计"）会漏检；升级向量检索是明确方向
3	AI 输出只有"格式校验"没有"内容校验"	四道防线保证 JSON 可解析、引用编号真实，但无法判断"顺延 3 天"是否真的合理；目前靠用户确认兜底
4	扫描版 PDF 不支持	纸质扫描文献无法入库，需接入 OCR
5	前端无自动化测试	回归依赖人工 + 后端冒烟；无 Vitest/Playwright
6	风险阈值硬编码	"落后 20 个百分点"等阈值集中在 config.py 但无 UI，无法按个人节奏调整
7	看板无拖拽、无提醒通知	MVP 决策使然，操作效率与风险触达有限
8	SQLite 单机文件库 + 开发环境双 venv	无并发写入/备份；双 venv 曾导致"后端无法加载"事故，根因是环境管理习惯

9.2 未来优化方向（按优先级）

P0（高价值、低成本）：① 文献切片做 embedding 向量检索（替代/混合关键词打分，解决语义召回弱）；② 重规划输出增加规则级合理性检查（如"调整后总工时是否仍超预算、是否引入新日期冲突"），让内容级风险尽量在入库前暴露；③ 风险阈值配置化到设置页。

P1（体验提升）：④ 拖拽看板（dnd-kit，保留按钮降级）；⑤ 本地托盘提醒 + 开机自启 + "今日该做什么"摘要；⑥ 前端自动化测试（Vitest + Playwright 关键路径 E2E）；⑦ 汇报中插入任务进度图表（数据已在库）。

P2（规模与健壮性）：⑧ 多用户与鉴权（JWT + 数据隔离），SQLite → PostgreSQL；⑨ Docker 化部署，消除 venv/node 环境问题；⑩ OCR 支持扫描版 PDF；⑪ DeepSeek JSON Mode/Function Calling 成熟后迁移（自建防线保留作兜底）；⑫ 全库数据一键导出/导入防丢失。

9.3 结语

ResearchPilot 的完整演进过程（从需求文档到 5 次提交、36 项测试、3 组 Prompt 案例）展示了一条可行的 **AI 协同开发路径**：人负责意图与边界，Claude 负责工程实现，DeepSeek 负责业务内容，代码层防线负责兜底。

第 3 章总结的意图控制策略在本项目中的有效性已被数据验证——它不是"大模型时代的玄学"，而是一套可以复制到下一个项目的工程方法。

ResearchPilot 研途智控 —— 科研任务智能管理，让每一步推进都有据可依。